ProofPay: Agentic Escrow and Revenue Recovery on PayPal

PayPal AI Hackathon merged submission document (PayFlow AI + ProofPay). Team: Antigravity. Prepared 30 Oct 2026. Working name: ProofPay (rename freely).

1. One-Line Pitch

ProofPay is a PayPal-native agent system that prevents non-payment with milestone escrow released on verified work, and recovers money when payment still fails, with every agent action bounded by a policy engine, explained, and auditable.

Core design rule (shared by both original projects): **the LLM proposes, the policy engine decides, and code executes.** No LLM ever calls PayPal directly.

2. Event Facts and Constraints

Item	Detail
Event	PayPal AI Hackathon (Devpost)
Deadline	Nov 12, 2026, 3:00 pm EST = Nov 13, 1:30 am IST. Treat Nov 12 evening IST as the real deadline
Prize pool	$67,500+ cash; grand prizes $12k / $8k / $5k, five $5k honorable mentions, sponsor prizes
Core rule	Meaningfully use PayPal (sandbox) and AI; working demo, public GitHub repo with license, YouTube video under 3 min
Judging	Five equal criteria: Technological Implementation, Design, Potential Impact, Innovation, Presentation
Prize limit	A project can win at most one Grand or Honorable prize plus one Sponsor prize
Team constraint	Zero-cost infrastructure; prefer scope cuts over time cuts

Sponsor pages were not reviewed when the source plans were drafted. Confirm each sponsor prize's requirements on Devpost before committing (see section 12).

3. Unified Problem Statement

Freelancers, small agencies and small merchants, especially in emerging markets, lose money at three points of the same payment lifecycle:

1. **Before work starts:** clients refuse to pay upfront because they cannot verify the work, and freelancers refuse to start without payment.
2. **At payment time:** subscriptions fail (expired card, insufficient funds, temporary hold) and invoices go overdue.
3. **After non-payment:** recovery is blind. Merchants retry every card the same way, send the same reminder to everyone, chase customers who already paid elsewhere (triggering disputes), and cannot explain why money was not recovered.

Existing escrow tools need a human to approve every release and disputes take weeks. Existing recovery tools report the problem but do not close the loop. Nobody connects prevention and recovery under one trustworthy, auditable control layer.

4. Unified Solution

One platform, two engines, one shared guardrail layer.

Engine	Origin	Job
Escrow Engine (Prevent)	ProofPay	Scope the job into milestones, hold funds, verify evidence with an agent, release via PayPal Payouts
Recovery Engine (Recover)	PayFlow AI	Detect failed or overdue payments, diagnose cause, choose the right intervention, stop at the right time, report money recovered
Shared Trust Layer	Both	Mandate/policy engine, reconciliation gate, dispute agent, append-only audit log, reputation ledger, dashboard, read-only Q&A

How the two engines connect (the real merge, not a bolt-on)

- **Funding failure feeds Recovery.** If a client's milestone funding payment or a split instalment fails, the case goes to the Recovery Engine with the same diagnosis and ladder.
- **Un-escrowed work feeds Recovery.** A freelancer's ordinary overdue PayPal invoice (work done outside escrow) is chased by the Recovery Engine, and the agent suggests escrow for the next project.
- **Disputes halt everything.** One dispute webhook halts the Recovery case and freezes the matching escrow milestone.
- **Reconciliation protects both sides.** Before any chase or any release, the gate checks whether the money already moved through PayPal, bank or merchant records.
- **Reputation is shared.** Releases, on-time payments, recoveries and disputes all feed one portable trust score per freelancer and client.

End-to-end lifecycle

```
JOB INTAKE -> SCOPING AGENT (milestones + checkable criteria) -> both sides
approve
 -> CLIENT FUNDS (PayPal Checkout / Orders v2) -> funds held in ledger
 -> FREELANCER SUBMITS EVIDENCE -> VERIFIER AGENT scores vs criteria
      score high  -> MANDATE GUARD -> PayPal Payout (via MCP / Agent Toolkit)
      borderline  -> client review with rationale
      low / none  -> deadline logic, reminder, dispute path
 -> FUNDING OR INVOICE FAILS at any point -> RECOVERY ENGINE
      detect -> already paid? -> diagnose -> prioritize -> strategy -> policy
check
      -> execute -> observe -> stop or escalate -> report
 -> DISPUTE ANYWHERE -> HALT + evidence pack -> DISPUTE AGENT
 -> every step -> AUDIT LOG -> REPUTATION LEDGER -> DASHBOARD
```

5. Target Users and Impact

- **Primary:** Indian and South Asian freelancers and small agencies billing foreign clients (the strongest "Most Impactful" story).
- **Secondary:** small online merchants with subscriptions and invoices (the PayFlow example: 120 subscribers, 40 invoice clients).

- **Why PayPal benefits:** more payment volume (recovered and escrowed payments), merchant retention, more PayPal products used (Checkout, Payouts, Subscriptions, Invoicing, Pay Later, Payment Links), fewer disputes, lower support load, and a credible model for AI agents handling money.

6. Features by Module

6.1 Escrow Engine

1. **Scoping agent.** Client describes the job in plain text. The agent outputs milestones, each with amount, deadline and machine-checkable acceptance criteria. Both sides approve.
2. **Funding.** Client pays once via PayPal Checkout (JS SDK v6, Orders v2). Funds are tracked against the milestone plan in a ledger.
3. **Verifier agent.** Freelancer submits evidence (GitHub PR, deployed URL, file). The agent scores it against criteria and writes a rationale. Above threshold: auto-release. Borderline: client review with reasoning attached.
4. **Release.** PayPal Payouts sends the milestone amount through the PayPal MCP server / Agent Toolkit, so the agent transacts instead of recommending.
5. **Timers.** Render cron handles auto-release timers and stale-escrow reminders.

Keep acceptance criteria concrete (tests pass, URL returns 200, file matches spec) so the demo never depends on a vague judgment.

6.2 Recovery Engine

1. **Detect** failed subscription payments and overdue invoices from PayPal webhooks and scheduled scans.
2. **Diagnose** cause: expired card, insufficient funds, temporary hold, forgot to pay, disputed, hard decline. Rules first, LLM only for messy text.
3. **Prioritize** by amount x recovery likelihood.
4. **Choose intervention** from the rules table below.
5. **Execute** under policy limits with a fixed escalation ladder and contact cap.
6. **Stop or escalate** on paid, dispute, opt-out or cap reached.
7. **Report** one headline number: recovered $X of $Y at risk across N cases, with reasons for the rest.

Interventions: scheduled retry, payment-method update request, invoice reminder, fresh payment link, split or Pay Later offer, promise-to-pay tracking, human escalation. Outreach is multilingual, polite and template-based.

Diagnosis	Action
Insufficient funds	Retry at payday-aligned timing, short reminder
Expired card	Email asking for a payment-method update
Temporary hold	Wait, then retry once
Forgot to pay	Friendly reminder, then payment link
Large overdue invoice	Offer split or Pay Later
"I'll pay Friday"	Log promise to pay, remind the day before
Dispute or opt-out	Stop immediately
Suspicious hard decline	No retry, escalate to a human

6.3 Reconciliation Gate (shared)

- Match PayPal, merchant and bank records by exact ID, then amount + date window + customer, then fuzzy match (rapidfuzz).
- Confidence tiers: auto-match, human review, exception.
- Close cases as PAID_ELSEWHERE with zero contact.
- Also verifies escrow funding and payouts against PayPal transaction records.
- Reports match rate, precision and recall against the hidden answer key, plus an exception list with a reason per record.

6.4 Mandate Guard / Policy Engine (shared, in code)

One deterministic allow/deny layer checks every write action from either engine.

Rule	Example value
Max amount per escrow release	Configurable per project
Auto-action amount limit	$200; above this needs human approval

Rule	Example value
Payout recipient	Verified accounts only
Maximum contacts per recovery case	4
Quiet hours	9 pm to 8 am customer time
Minimum gap between contacts	48 hours
Daily action cap	Configurable
LLM budget	Max 2 calls per case, daily cap
Hard stops	Dispute opened, opt-out, payment received, cap reached
Allowed channels	Email and payment link

A policy block is logged and escalated to a human, never bypassed.

6.5 Dispute Agent (shared)

PayPal dispute webhooks halt the related recovery case and freeze the escrow milestone. The agent assembles an evidence pack from submission history, verifier rationale, contact history and timestamps. It proposes; a human decides.

6.6 Reputation Ledger

Every release, on-time payment, recovery and dispute updates a portable trust score for freelancers and clients. Scores are transparent: each change links to its audit entry. This is the team's own addition beyond the PayPal integration.

6.7 Observability, Dashboard and Q&A

- Append-only audit log: timestamp, case, actor, decision, one-line reason, inputs used, policy checks passed, PayPal ID, outcome.
- Dashboard (React + AG Grid): headline metrics, escrow and recovery ledger with master-detail rows, case timeline, reconciliation view, unrecovered/halted views.
- Searchable audit trail and failure-reason charts.
- Read-only Q&A chat over audit and case data, citing the records used (for example "Why wasn't case 42 recovered?" or "Why was milestone 2 held?").
- Evaluation harness comparing the agent with a naive baseline.

7. State Machines

Escrow milestone: DRAFT -> PLAN_APPROVED -> FUNDED -> SUBMITTED -> VERIFYING -> (RELEASE_PENDING_APPROVAL if over limit) -> RELEASED | IN_REVIEW | DISPUTED | REFUNDED.

Recovery case: OPEN -> RECONCILING -> DIAGNOSED -> ACTION_PENDING_APPROVAL (if large) -> ACTION_SENT -> AWAITING_RESULT -> RECOVERED | HALTED | UNRECOVERED | PAID_ELSEWHERE.

Terminal states are final. A case cannot reopen without a human action, which makes the stopping rules provable.

8. Data and Simulation

The PayPal sandbox cannot simulate real customer behaviour or real payment failures. The project uses synthetic data with a hidden answer key, real sandbox calls for actions, and simulated webhook events for failures.

8.1 Datasets

Dataset	Source
Customers (id, email, language, opt-out, history)	Generated
Subscription events (plan, amount, failure reason, timestamps)	Generated, injected as simulated webhooks
Invoices (amount, due date, days overdue, status)	Generated and created in PayPal sandbox
PayPal transactions	Generated in PayPal format and sandbox
Merchant sales records (CSV) and bank statement (CSV)	Generated
Escrow projects, milestones, submissions and evidence	Seeded demo data
Customer behaviour profiles	Simulator, hidden

8.2 Scale

150+ recovery cases (about 90 failed subscriptions, about 60 overdue invoices), 200+ reconciliation records across three sources, and 10+ seeded escrow projects with varied verification outcomes.

8.3 Planted traps

- Customers who already paid by bank transfer (key test of the gate)
- Duplicate payments and partial refunds
- Fee deductions and currency rounding gaps (for example 0.03)
- Timing offsets between PayPal and the bank
- Customers who dispute if chased too early
- Hard declines that look like fraud
- Customers who promise to pay and then do or do not
- Evidence submissions that nearly meet criteria (borderline verifier cases)
- Over-limit release requests (mandate guard test)

8.4 Hidden customer profiles

Profile	Behaviour
Pays after one reminder	Recovers on first friendly contact
Needs a payment link	Ignores email, pays when a link is sent
Needs card update	Pays only after a payment-method request
Prefers split or Pay Later	Pays only when offered instalments
Ignores everything	Never pays; should be escalated early
Disputes if chased too early	Opens a dispute after first contact
Already paid elsewhere	Paid by bank; any contact is a mistake

Payment probability depends on how well the intervention matches the hidden true cause. All assumptions are visible in the UI and the submission.

9. Architecture

9.1 Layers

```
DATA SOURCES: PayPal APIs + Webhooks | Merchant CSV | Bank CSV | Simulated events | User submissions
 -> INGESTION: verify signature, dedupe by event ID, normalize, event queue
 -> POSTGRESQL: customers, projects, milestones, cases, transactions, attempts, evidence, audit log, reputation
 -> SCHEDULER: timed retries, reminders, promise-to-pay checks, escrow auto-release timers
 -> GATE 1: RECONCILIATION (already paid? funding confirmed?)
 -> INTELLIGENCE: scoping, verifier, diagnose, prioritize, strategy, dispute (rules first, LLM for messy cases)
 -> GATE 2: MANDATE GUARD / POLICY ENGINE
 -> EXECUTION: PayPal Checkout, Payouts, Invoicing, payment links, Pay Later, email (simulator answers in demo mode)
 -> VERIFICATION & AUDIT: confirmation, reconciliation re-check, audit entry, search index
 -> OUTPUTS: React dashboard + AG Grid, Q&A chat, evaluation harness, reputation view
```

9.2 Components

Component	Responsibility	Technology
Web app	Intake, plan approval, funding, submissions, ledger	Next.js / React + AG Grid
Webhook receiver	Verify, dedupe, persist events fast	FastAPI
Event queue	Decouple receiving from processing	Postgres table (Redis optional)
Reconciliation service	Match records with confidence tiers	Python, pandas, rapidfuzz
Agents	Scoping, verifier, recovery, dispute with tool calling	LLM API with tool schemas
Mandate guard	Deterministic allow or deny	Plain code + config table

Component	Responsibility	Technology
Executor	PayPal calls, emails, retries, payouts	PayPal REST client / MCP / generated SDK
Scheduler	Timers, retries, reminders	APScheduler or Render cron
Simulator	Hidden profiles that respond	Python module
Audit service	Append-only log, searchable	Postgres (+ Elastic if used)

Use one backend language (Python) to avoid doubling deployment and debugging work.

9.3 Agent tools

Tool	Type	Gated by policy
get_case, get_customer_history, get_milestone	Read	No
check_already_paid	Read	No
score_submission	Read (LLM)	No
schedule_retry	Write	Yes
send_invoice_reminder	Write	Yes
create_payment_link	Write	Yes
offer_split_payment	Write	Yes
log_promise_to_pay	Write	Yes
release_milestone_payout	Write	Yes
freeze_milestone	Write	No
escalate_to_human	Write	No

9.4 Data model (minimum)

- customers: id, email, language, opt_out, notes
- projects: id, client_id, freelancer_id, total, status
- milestones: id, project_id, amount, deadline, criteria, status
- submissions: milestone_id, evidence_ref, score, rationale, timestamp

- cases: id, type, amount, status, diagnosis, priority, created_at
- attempts: case_id, action, channel, timestamp, result, paypal_id
- reconciliation_matches: case_id, source records, confidence, tier
- audit_log (append-only): timestamp, case_id, actor, decision, reason, policy checks, inputs, outcome
- reputation: subject_id, score, event_refs
- policy_config: limits, quiet hours, thresholds, budgets

9.5 Failure handling

Failure	Handling
Duplicate webhook	Dedupe by event ID, process once
PayPal API error	Retry with backoff, then mark ACTION_FAILED and escalate
LLM empty or bad output	Fall back to rule table, log the fallback
Policy blocks an action	Log, escalate to a human, never bypass
Amount mismatch on retry or payout	Stop and flag for review
Payouts not enabled on sandbox	Fall back to Orders and Invoicing for release

10. PayPal Resources Used

PayPal resource	How it is used
Developer Dashboard and Sandbox	Business and buyer accounts, credentials, webhook registration
REST API with OAuth	Auth for all calls, token cached until expiry
Checkout + Orders v2 (JS SDK v6)	Client funding of escrow, fresh payment links
Payouts	Milestone release to freelancer
Agent Toolkit and MCP server	Tool layer for the agents, wrapped by the mandate guard

PayPal resource	How it is used
Subscriptions	Plans, failure events, suspend/activate/cancel, outstanding balance capture
Invoicing	Create/send invoices, reminders, record payments, paid/overdue events
Webhooks	Real-time detection, stopping rules, signature verification, dedupe
Disputes	Dispute events halt cases and feed the "disputes avoided" metric
Pay Later	Split or instalment offer for large overdue invoices
Payment links and buttons	Low-code payment links in outreach
PayPal AI docs	Confirm supported agent protocols and tool references

Confirm exact endpoint and event names in PayPal docs, since they change. Real sandbox calls are used for actions; failure scenarios the sandbox cannot produce are injected as simulated webhooks.

11. Wow Factors

1. **One headline number:** "Recovered $X of $Y at risk across N cases" is the first thing on screen.
2. **Money moves only on proof:** agent-verified milestone release through the PayPal MCP server, guarded by a mandate.
3. **Never chase someone who already paid:** reconciliation gate before any outreach (target zero wrongful contacts).
4. **Provable stopping rules:** a dispute halts a case permanently and freezes the milestone, with the audit trail on screen.
5. **Agent vs baseline, measured:** same 150 cases through "retry three times, same email" and through the agent.
6. **Mandate guard refuses live:** an over-limit release is blocked on camera.
7. **Honest exception list:** every unrecovered case and every refused match has a reason.

8. **Cost-aware:** rules first, templates, cheap model for leftovers; report share of cases with no LLM call.
9. **Pay Later as a recovery tool:** promotes a PayPal product directly.
10. **Reusable guardrails layer:** the policy engine is packaged so any developer can wrap it around the PayPal Agent Toolkit / MCP server.
11. **Portable reputation:** trust score built from verified behaviour.

12. Sponsor Integrations and Prize Strategy

A project can win at most one Grand/Honorable prize plus **one** sponsor prize. Integrate several sponsors where each has a real job (this also helps Technological Implementation), but concentrate the polish on one sponsor prize.

12.1 Sponsor map

Sponsor	Real job in the merged project	Priority
AG Grid	Unified ledger: escrow milestones, payouts, recovery cases and disputes in one live grid with master-detail rows, filters (status, intervention, amount), "unrecovered" view, row click to audit trail, webhook-driven live updates, audit export	**Must (first-choice sponsor prize)**
Render	Hosts web app, API/webhook receiver, worker, Postgres and cron; ship a render.yaml blueprint; public URL registered as PayPal webhook endpoint; judges get a live link	**Must (fallback sponsor prize)**
Postman	Shared collection of PayPal calls (orders, payouts, invoice, reminder, retry) plus webhook-simulation folder with automated tests so judges can reproduce the flow	High
Elastic	Index audit events for full-text search of any case or milestone history and failure-reason charts	High (only if free tier fits)
APIMatic	Generate a typed PayPal SDK from the OpenAPI spec, use the context plugin while building, document in README	Medium (cheap, concrete)

Sponsor	Real job in the merged project	Priority
Bryntum	Gantt of milestones with payment status per bar; timeline of each customer's escalation ladder and promise-to-pay dates	Stretch: skip if behind on day 8
Zapier	Outreach actions (email, Slack alert) and human-approval step for large actions; core logic stays in our code	Optional

Not used: Channel3 (shopping search, not relevant). Check KERNEL and Astropods Devpost pages and add only if a specific job exists.

12.2 Prize targeting

Prize	Angle
Best Use of Agentic Commerce / PayPal + AI	Agents release real sandbox payments through MCP with mandate guardrails; recovery agent acts under hard limits
Most Impactful	Freelancer non-payment story for Indian and South Asian freelancers billing foreign clients, plus merchant revenue recovery
1st to 3rd Place	All five criteria: technical depth (PayPal flows, verifier, reconciliation), polished UI, complete flow, tight 3-minute video
Sponsor prize (one)	AG Grid first choice, Render fallback

12.3 Rule to confirm before committing

Verify each sponsor's specific prize requirements on Devpost (for example required usage depth, extra deliverables, separate submission fields). Name every AI tool and sponsor tool and how each is used in the description.

13. Zero-Cost Infrastructure Plan

- **Hosting:** Render free tier for web service, worker and Postgres if limits allow; otherwise the Oracle Cloud Always Free VM with NGINX, Uvicorn, PostgreSQL, DuckDNS and Let's Encrypt. Confirm current free-tier limits (sleep on idle, database expiry) before demo day, and keep the demo awake before recording and judging.
- **Elastic:** use only if a free tier or trial covers judging; otherwise Postgres full-text search keeps the audit trail searchable at zero cost.
- **LLM:** cheap model for routine work, larger model only for Q&A; hard per-case and per-day call caps enforced by the policy engine; templates generated once.
- **Channels:** email as the default; PayPal built-in invoice reminders where possible; Telegram Bot API as an optional free alert channel.
- **CI:** GitHub Actions; single `main` branch with folder ownership per role.
- **PayPal:** sandbox only (free).

14. Evaluation and Metrics

The same cases run through two strategies: the baseline (retry three times, same email to all) and the agent.

Metric	Why it matters
Recovered $ / at-risk $ and recovery rate	Headline number
Recovery by intervention type	Shows the agent picks the right action
Contacts per recovery	Lower means less annoyance and cost
Customers wrongly contacted (already paid)	Target zero vs baseline count
Disputes avoided	Value to PayPal and merchant
Reconciliation precision and recall	Measured against hidden answer key
Verifier accuracy on seeded submissions	Agreement with planted pass/fail/borderline labels
Unsafe actions blocked by mandate guard	Proves guardrails work

Metric	Why it matters
Share of cases handled without an LLM call	Cost control
Estimated cost per recovered dollar	Labeled as an estimate
Unrecovered cases with reasons	Honest reporting

15. Demo Video Outline (under 3 minutes)

Time	Beat
0:00 - 0:20	Problem with one real freelancer scenario: client won't pay upfront, then pays late
0:20 - 1:00	Describe a job; milestone plan appears; client funds via PayPal Checkout
1:00 - 1:50	Freelancer submits work; verifier scores it; payout fires via MCP; AG Grid ledger updates live
1:50 - 2:20	Mandate guard refuses an over-limit release; a dispute halts the case and shows its evidence pack
2:20 - 2:45	Recovery headline: recovered $X of $Y vs baseline; already-paid customer got zero contact
2:45 - 3:00	Reputation score, stack and sponsor summary, impact line

Requirements: shows the app working end to end on its target device, public on YouTube, no copyrighted music or third-party trademarks. Cut the Q&A demo from the video if time runs over; keep it in the live app.

16. Build Plan (13 days from Oct 30)

Days	Dates	Focus
1 - 2	Oct 30 - 31	Sandbox app, Checkout + Orders, Payouts test, webhook receiver, repo, LICENSE, render.yaml skeleton, Postgres schema
3 - 5	Nov 1 - 3	Scoping agent, milestone model, mandate guard, MCP-based release, synthetic data generator + simulator
6 - 8	Nov 4 - 6	Verifier agent, webhook handling, recovery agent loop for failed subscriptions, AG Grid ledger
9 - 10	Nov 7 - 8	Invoice chaser, reconciliation gate, dispute agent, reputation score, baseline comparison (Bryntum only if ahead)
11	Nov 9	UI polish, seeded demo data, Postman collection, README with setup and test credentials
12	Nov 10	Record video, deploy, dry-run as a judge would
13	Nov 11	Submit with buffer; do not wait for the last hour

Scope-cut order (cut scope, not time)

1. Bryntum, Zapier, Elastic (use Postgres search)
2. Q&A chat beyond a basic cited answer
3. Subscription recovery depth (keep invoice chasing + reconciliation gate)
4. Reputation score sophistication (keep a simple transparent formula)

Never cut: escrow fund -> verify -> release path, mandate guard, dispute halt, audit log, AG Grid ledger, hosted demo, video.

Suggested role split

Frontend (Next.js + AG Grid), backend (FastAPI, PayPal, webhooks, scheduler), agents (scoping, verifier, recovery, dispute, policy engine, simulator). Lock the data model on day 1 and keep folder ownership per role.

17. Risks and Day-1 Checks

- **Payouts access:** confirm Payouts is enabled on the sandbox business account; else use Orders + Invoicing for release.
- **MCP tool names:** confirm what the PayPal MCP / Agent Toolkit can actually do in the quickstart before designing around it.
- **Verifier reliability:** concrete, checkable acceptance criteria only.
- **Scope inflation from the merge:** the Escrow Engine is the primary demo path; Recovery Engine is the second act. Protect the one-prize focus.
- **Free-tier limits:** test cold starts and database retention before judging.
- **Webhook reliability:** public URL early, signature verification and dedupe from day 1.

18. Honest Limits

- Recovery rates come from a customer simulator, not real customers; assumptions are visible in the UI.
- The sandbox cannot produce real failures, so failure events are simulated while actions use real sandbox calls.
- This complements PayPal's own recovery features and does not replace them.
- The policy engine follows configurable rules and is not legal compliance. Production use in collections or escrow would need legal and regulatory review.
- Cost and time-saved figures are estimates.
- Reputation scores are a prototype, not a credit score.

19. Submission Checklist

Item	Requirement
Repository	Public, all source and assets, open-source license visible in About
Working demo	Hosted URL, or full run instructions + sandbox credentials in README. Mockups do not count
Video	Under 3 min, public on YouTube, end-to-end on target device, no copyrighted music or third-party trademarks

Item	Requirement
Description	Features and functionality; name every AI tool and sponsor tool and how each is used; explain what was updated during the period if the project pre-existed
README	Easy to skim with clear structure (judging may include automated AI analysis); setup, test credentials, architecture, sponsor usage
Sponsor prize	Confirm requirements per sponsor on Devpost

20. Open Items to Confirm

- Team size and role strengths (for the final task breakdown)
- Final project name
- Which one sponsor prize to target (recommended: AG Grid, fallback Render)
- Payouts enablement and MCP capabilities (day 1)
- Free-tier limits for Render and Elastic